

一、SGLang 的 RadixAttention / RadixCache 解决什么问题？

RadixAttention 主要解决大模型推理中跨请求的 KV Cache 复用问题。在传统推理引擎中，每个请求结束后其 KV Cache 会被直接丢弃，即使多个请求包含相同的前缀，也无法复用。RadixAttention 将 KV Cache 组织为基数树结构，通过自动匹配最长公共前缀，实现跨请求的缓存复用。通过这个方法减少了重复的 Prefill 计算，降低了显存占用并且显著提升了系统吞吐量。

二、SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

page-sized KV cache 对应 pipeline 中的 KV Cache 管理环节。prefix reuse 对应 pipeline 中的 RadixCache 环节。

三、vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention/paged KV 机制有什么联系与差异？

vLLM 的 PagedAttention 解决 Cache 的显存碎片化与浪费问题。传统方法需要为每个请求预分配连续显存，导致严重的内存浪费。PagedAttention 借鉴操作系统虚拟内存分页思想，将 KV Cache 切分为固定大小的块进行散存储和按需分配，显存利用率显著提高。

联系：两者都受操作系统分页机制启发。SGLang 的 page-sized KV cache 直接吸收了 PagedAttention 的分页管理思想，解决了显存碎片问题。

差异：vLLM 的 PagedAttention 核心在于单请求内的显存高效分配，SGLang 的 RadixAttention 在此基础上构建了跨请求的基数树结构，支持更复杂的多轮对话分支、自动最长前缀匹配、LRU 淘汰等机制。